Prompt Classifier

Placeholder

November 2, 2026

---

1 Introduction

As a Linux user I sometimes find myself running command line interface (CLI) commands that I do not fully understand. Even if I look up the documentation for the commands it is not always clear what effect a command a will have on my specific machine. One alternative to parsing the documentation is to run the commands in a sandbox mirroring my system see what happens. Simple in theory but practically I ran into a problem, it was often impossible to verify whether a command worked as expected.

Consider the following questions that are among the 100 most viewed questions on `https://askubuntu.com/`:

1. How do I save terminal output to a file? (Verifiable Minimal)
2. How to list all installed packages? (Unverifiable Minimal)
3. How do I install Ubuntu alongside a pre-installed Windows with UEFI? (Unknowable)

Further lets assume that we have access to Ubuntu sandboxes, can we verify CLI commands satisfying these prompts? "How do I save the terminal output to a file?" can be verified by writing the terminal output to a file and checking that the files contents matches the terminal output. However it is not clear that the prompt "How to list all installed packages?" can be verified in a sandbox since it would require knowing all packages installed in the sandbox. Finally it is clear that no set of CLI commands run in a Ubuntu sandbox can satisfy the question "How do I install Ubuntu alongside a pre-installed Windows with UEFI?". This is a problem for any CLI command sandbox verification system since not all prompts or CLI command responses can be verified.

This project attempts to solve this problem by classifying prompts and proposed CLI command responses. The following section introduces a simple 8bit Adder system which is easier to reason about than Ubuntu and example prompts. We then outline 5 categories that we hope cover all possible system prompts and show that these prompts can be classified with a decider model like Cloudflare's Clef-Flash model. Finally we return to the three Linux prompts and show that a similar system can also be used to classify Linus prompts.

2 8bit Adder System

Lets imagine a simple 8bit adder service. The aim of this system was to be easy to reason about rather than useful. The 8bit Adders functionality is detailed below:

1. The adder is a micro service whose endpoints are accessible via routes of the form GET `http://localhost/endpoint`

2. The adder supports adding 8 bit non-negative numbers A and B at endpoint GET `http://localhost/add/{A}/{B}` and returns A+B = R if R is 8 bit or R=InvalidRequest if it is not.

3. The adder also supports getting the adder logs at endpoint GET `http://localhost/logs` which return all requests made to the service and the services responses as a json in the form request_id, timestamp, request, response.

4. All other requests to the service for the form `http://localhost/X` where X is any string including the empty string but excepting the routes detailed above return R=InvalidRequest.

3 8bit Adder Prompt Taxonomy

The prompt taxonomy I settled on for system prompts is:

- Unknowable: Prompts that cannot be answered by the system alone.
- Unverifiable: Prompts that cannot be verified by the prompter.
- Unverifiable Minimal: Prompts that cannot be verified by the prompter and whose system commands are the minimal set required.
- Verifiable: Prompts that can be verified by the prompter.
- Verifiable Minimal: Prompts that can be verified by the prompter and whose system commands are the minimal set required.

Below examples 8bit adder prompts, Spark Muse generated commands for the 8bit adder system, and my classification:

- How do I tell what language the system is written?
  Commands: GET `http://localhost/language`
  Classification: Unknowable
- How do I know the services log timestamps are accurate?
  Commands: GET `http://localhost/add/10/20`, GET `http://localhost/time`, GET `http://localhost/logs`
  Classification: Unverifiable
- How do I know the services log timestamps are accurate?
  Commands: GET `http://localhost/timestamps`
  Classification: Unverifiable Minimal
- How do I add 5 and 7 using the adder service?
  Commands: GET `http://localhost/add/5/7`, GET `http://localhost/logs`
  Classification: Verifiable
- How do I add 5 and 7 using the adder service?
  Commands: GET `http://localhost/add/5/7`
  Classification: Verifiable Minimal

4 8bit Adder Prompt Classification

The prompt classifier queries the Cloudflare decider model Clef-Flash with either 1 or 3 of the questions below following the flow in Figure 1.

- Q1: Is the prompt answerable by the system according to its specification?
- Q2: Does the prompt ask the system to perform an action with a checkable result, and not about things that require perfect knowledge of the system (such as whether the system logs are complete or accurate, what other users did, timestamp accuracy, etc.)?

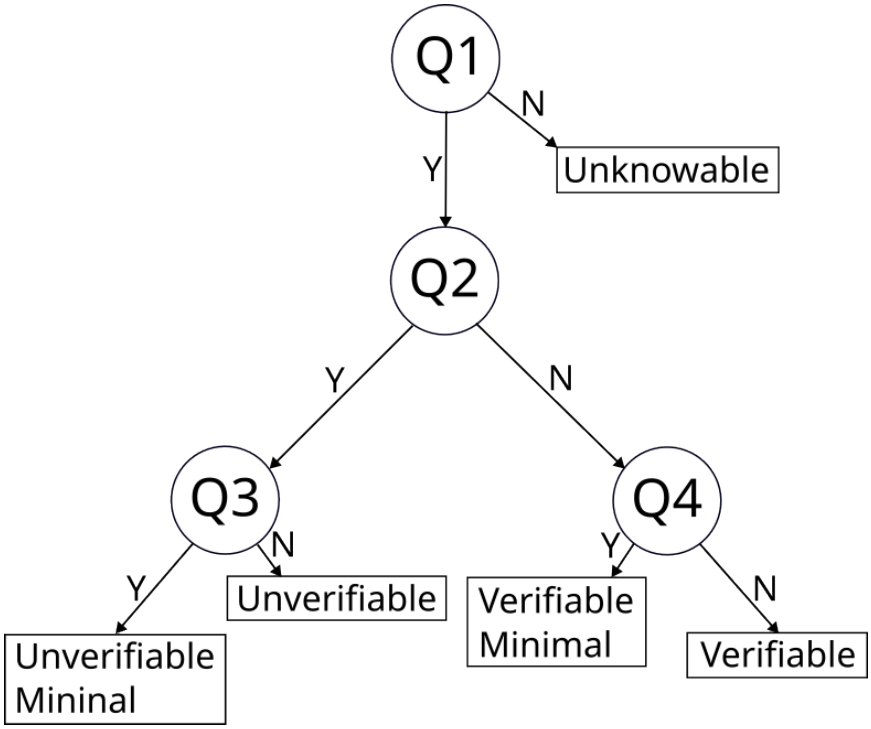


Figure 1

- Q3, Q4: Given the system specification is every request in the LLM_response required for the prompt's task?

Clef-Flash returns the answer to the question (Y/N) and a probability for the answer. With Muse Code's help I tuned both the questions and the decision cutoff points for the questions in order to achieve the following results. I think this tuning likely over fit the 8bit Adder prompt data set which consists of only 50 prompts, 10 in each category. However my goal here was not to have this prompt classification framework generalize but only to show that it can work, and I hope the following results show that.

On the set of 8bit Adder prompts in shuffled_prompts_4.csv the Clef-Flash model using the question tree in Figure 1, the questions above, and the cutoff points $\{q1 : 0.07, q2 : 0.82, q3 : 0.3, q4 : 0.6\}$ achieved a categorization accuracy of 40/50 (80.0%). The per category accuracy, precision, and recall are given in Table 1.

Category	**Accuracy**	**Precision**	**Recall**
Unknowable	10/10 (100.0%)	0.62	1.00
Unverifiable	9/10 (90.0%)	0.90	0.90
Unverifiable Minimal	6/10 (60.0%)	0.86	0.60
Verifiable	8/10 (80.0%)	1.00	0.80
Verifiable Minimal	7/10 (70.0%)	0.78	0.70

Table 1: 8Bit Adder Prompt Classification Results

The confusion matrix for the prompt classification of the 8bit Adder prompts is given in Figure 2.

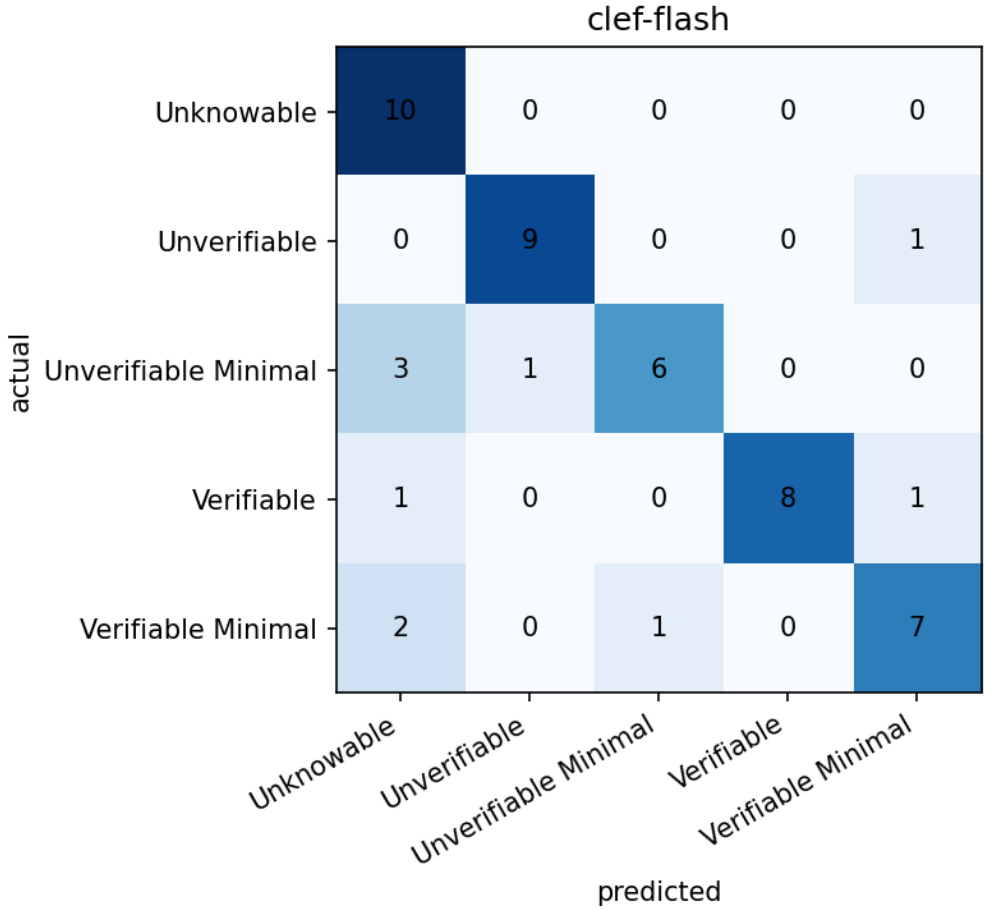


Figure 2: 8Bit Adder Prompt Classification Confusion Matrix

5 Ubuntu Prompt Classification

Given that the classification results for the 8bit Adder prompts appeared to work we decided to see if the the classification would also work on the 100 most viewed questions on `https://askubuntu.com/`. For prompts we did not use the questions themselves which can be lengthy but the question titles are usually more focused. We used Muse Spark to get command responses for the prompts and then I labeled them giving linus_question_responses_groundtruth.csv. The fact that I labeled the Linux prompt is a weakness because my labeling might be incorrect. Still for a POC I think the following results do suggest that classifying Linux prompts is possible even if not all the ground truth classifications are necessarily accurate.

Again I used Muse Spark to tune the probability cutoffs and the phrasing of Q2 which depends on what actions the system takes which can and cannot be verified. In the future we would like to find a phrasing of Q2 that generalizes better across systems. The system definition, Q2, and the cutoffs we settled on where:

- System Definition: The system is a Linux OS Ubuntu distribution version 26.04.
- Q2: Does the prompt ask the system to perform an action with a checkable result (modifying an application, file, or user), and not for information only the system itself knows (package list, system version, users, logs, etc)?
- Cutoffs: $\{q1 : 0.21, q2 : 0.11, q3 : 0.08, q4 : 0.25\}$

Using these parameters the categorization accuracy of Linux prompts was 79/100 (79.0%), while impressive at first glance this high degree of accuracy is largely due to most of the prompts in the ground truth set being classified as Verifiable Minimal. The per category accuracy, precision, and recall are given in Table 2.

Category	Accuracy	Precision	Recall
Unknowable	1/5 (20.0%)	0.20	0.20
Unverifiable	0/0 (n/a)	0.00	n/a
Unverifiable Minimal	12/14 (85.7%)	0.60	0.86
Verifiable	0/1 (0.0%)	0.00	0.00
Verifiable Minimal	66/80 (82.5%)	0.96	0.82

Table 2: Linux Prompt Classification Results

The confusion matrix for the prompt classification of the Linux prompts is given in Figure 3.

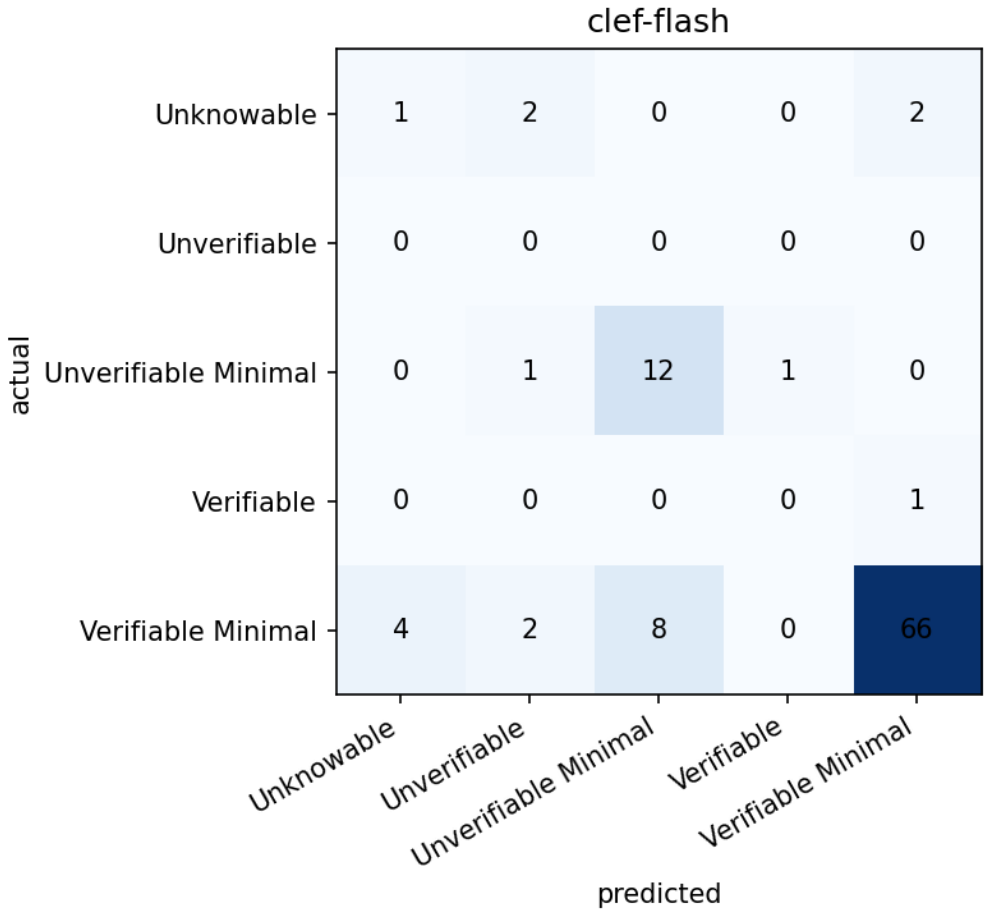


Figure 3: Linux Prompt Classification Confusion Matrix

Perhaps a better classification to test moving forward given that most popular questions on `https://askubuntu.com/` and the set of commands generated by Muse Spark are Verifiable Minimal, is classifying prompts and command sets as either Verifiable Minimal or not. This simpler taxonomy is has some justification in that Verifiable Minimal command sets can in theory be verified in a sandbox while all other classifications cannot.

The accuracy, precision, recall, and specificity of the Verifiable Minimal prompt classifier are:

- Accuracy: 83.0%
- Precision: 0.96
- Recall: 0.82
- Specificity: 0.85

with 66 True Positives, 3 False Positives, 14 False Negatives, and 17 True Negatives. Meaning a model like this could be used with some effectivness to classify prompt and command set pairs that are not Verifiable Minimal at the cost of miss classifying about 17.5% of Verifiable Minimal prompts.